



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 1: Introduction to Computational Intelligence and Nature-Inspired Algorithms

Dr Pedro Machado pedro.machado@ntu.ac.uk

Introduction

Module Leader:

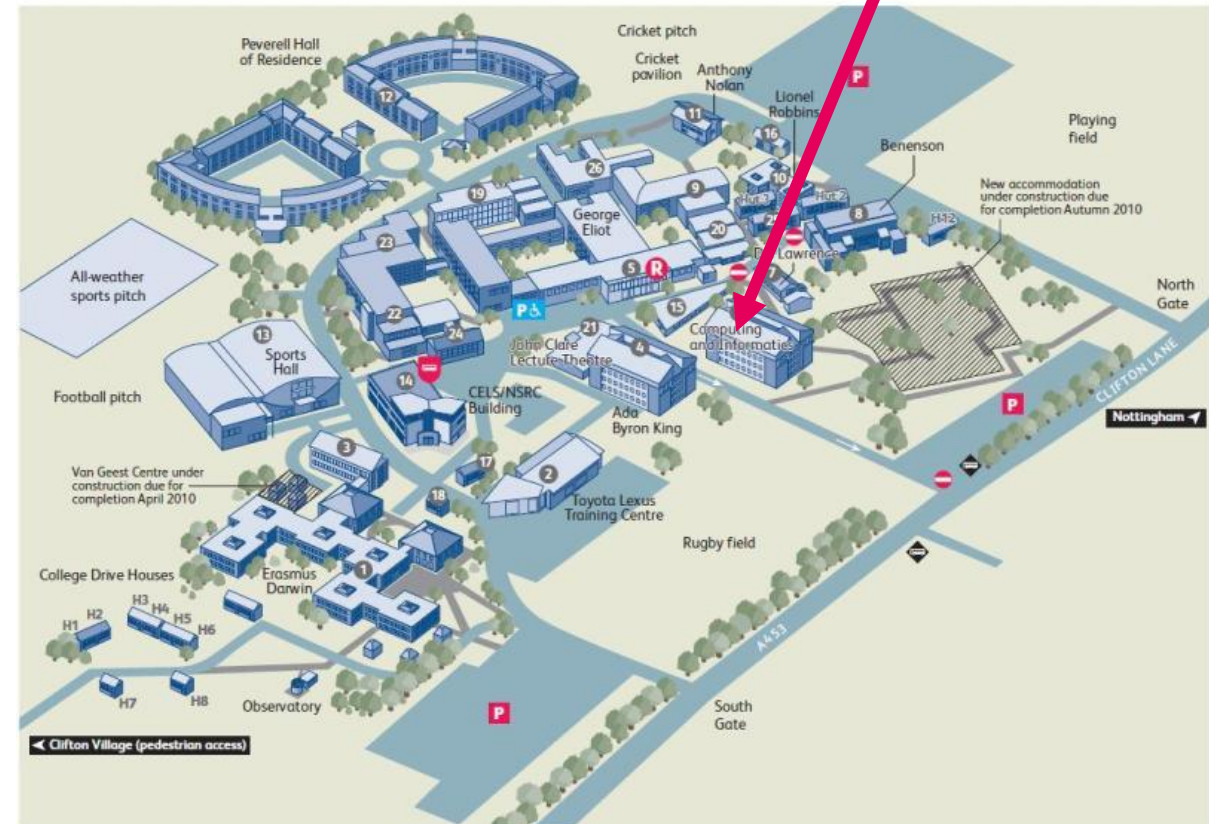
Dr Pedro Machado

Senior Lecturer in Computer Sciences

email: pedro.machado@ntu.ac.uk

Office Hours: Monday 09:00 – 11:00 in MAE326A.

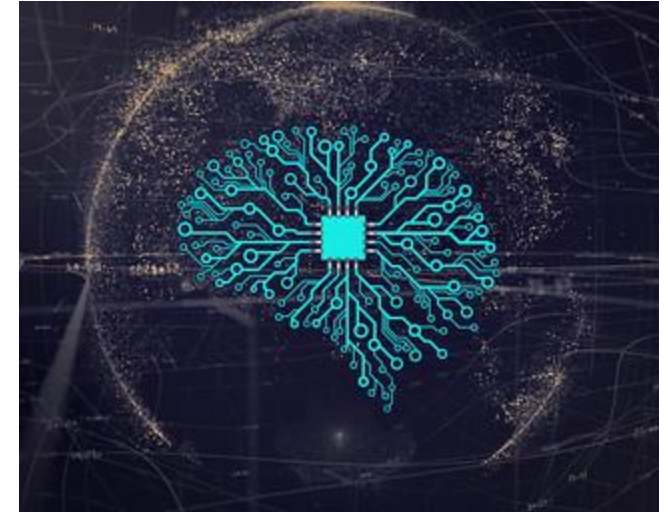
Clifton campus



Computational Intelligence

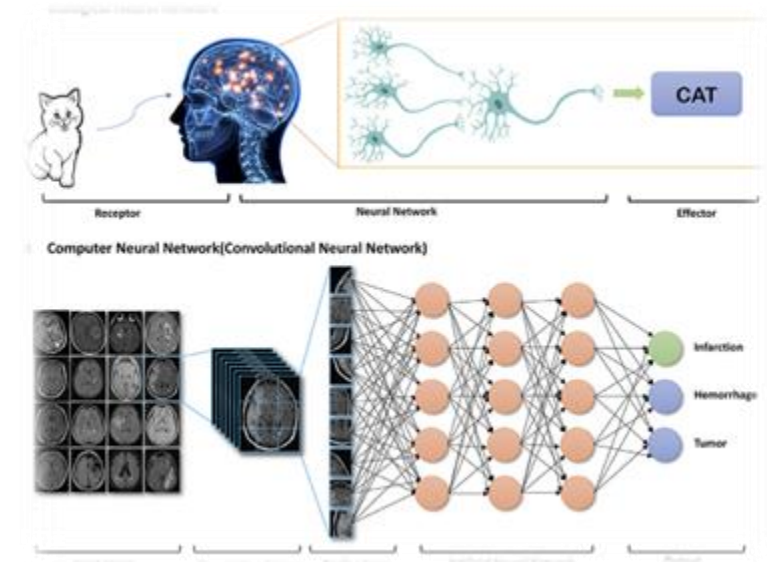
The module will cover the following concepts :

- Nature-Inspired Optimisation
- Brain-Computer Interfaces
- Image Processing and Analysis
- Data Mining and Machine Learning
- Reinforcement Learning



Assessment

- Your success in Artificial Cognitive will be tested and assessed by:
 - **Formative** Class Test: Online 8 questions class-test. **[w/c 02 February 2026]**
 - Mandatory live demo. Submission of 6-slides presentation and source code. **[submission deadline: 23 March 2026 by 2:30pm]**





Nottingham Trent
University

Department of Computer Science

Lecture Outline

In this lecture will cover:

- What is Computational Intelligence (CI) and how it relates to AI
- Core principles of CI
- Nature-Inspired Optimisation
- Swarm Intelligence (PSO, ACO, ABC)
- Evolutionary Computation (GA, ES, DE)
- Hybrid CI concepts
- Discussion of real-world optimisation scenarios

This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.

What is Computational Intelligence?

Computational Intelligence (CI) is a branch of AI that uses adaptive, nature-inspired, and learning-based methods to solve complex problems.

Core traits:

- Adaptivity
- Robustness
- Emergent behaviour
- Learning from data or environment



Why Computational Intelligence?

- Traditional algorithms struggle with uncertainty, non-linearity, and incomplete data.
- CI provides flexible, human-like problem-solving strategies for real-world challenges such as optimisation, pattern recognition, and decision-making.

Artificial vs Computational Intelligence

Artificial Intelligence (AI):

- Logic-driven, symbolic reasoning
- Knowledge-based systems

Computational Intelligence (CI):

- Bio-inspired, adaptive, and data-driven
- Focuses on learning and evolution



Comparison chart

ARTIFICIAL INTELLIGENCE

VS

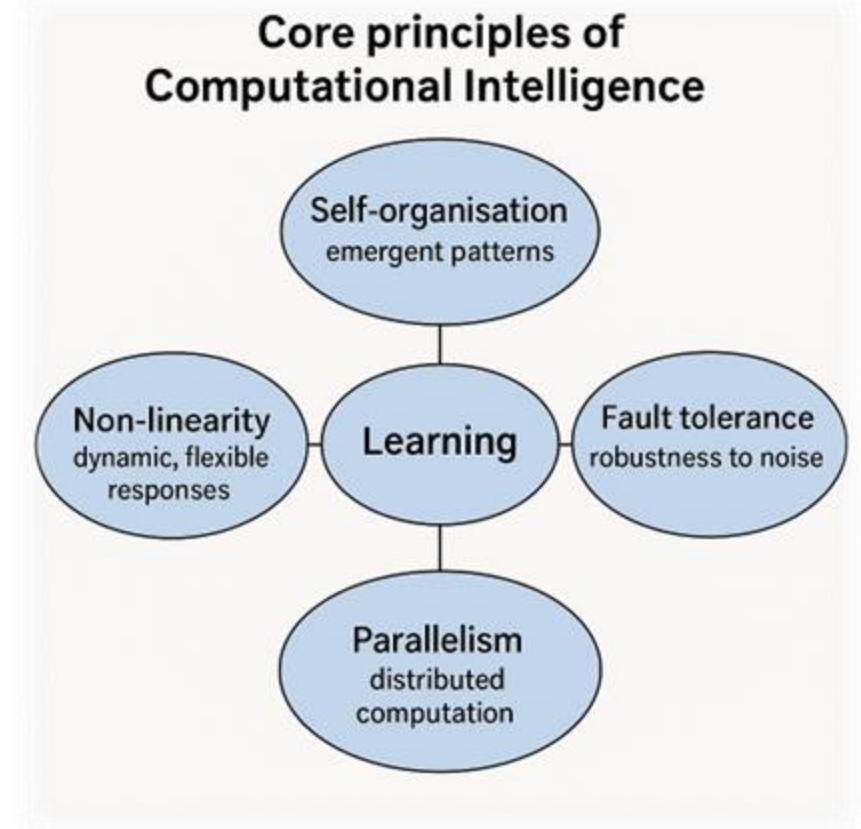
COMPUTATIONAL INTELLIGENCE

CI vs AI vs ML

Aspect	Artificial Intelligence (AI)	Computational Intelligence (CI)	Machine Learning (ML)
Primary Goal	Simulate human reasoning and decision-making	Solve complex, uncertain, dynamic problems using adaptive, nature-inspired methods	Learn patterns from data for prediction and classification
Core Methods	Symbolic logic, rule-based systems, expert systems	Evolutionary algorithms, swarm intelligence, fuzzy logic, neural networks	Statistical models, supervised/unsupervised learning, deep learning
Adaptability	Low - rigid rules, struggles with incomplete data	High - stochastic, adaptive, robust to noise and uncertainty	Moderate - adapts with retraining, but needs large datasets
Strengths	Precise reasoning, deterministic outputs	Handles non-linearity, multimodal search, dynamic environments	Strong predictive power with large data
Limitations	Poor scalability in uncertain domains	Parameter tuning complexity, slower convergence in some cases	Data-hungry, less interpretable
Example Where CI Wins	-	Real-time swarm-based traffic optimisation in smart cities; evolutionary scheduling in dynamic manufacturing	-

Core Principles of CI

- Learning: ability to adapt from experience
- Self-organisation: emergent patterns
- Fault tolerance: robustness to noise
- Parallelism: distributed computation
- Non-linearity: dynamic, flexible responses



Symbolic vs Sub-Symbolic AI

Scenario: A global e-commerce company needs to optimise delivery routes for thousands of packages across multiple cities under dynamic conditions (traffic, weather, last-minute orders).

Symbolic Planning Approach:

Method:

- Rule-based planning using deterministic algorithms (e.g., STRIPS, PDDL).

Strengths:

- Precise, interpretable plans when environment is static and fully known.

Limitations:

- Fails under uncertainty or when real-time adaptation is required.

Example:

- Traffic jams or sudden route closures require manual replanning.

Swarm-Based Optimisation Approach

Method:

- Ant Colony Optimisation (ACO) or Particle Swarm Optimisation (PSO) for route selection.

Strengths:

- **Decentralised, adaptive**, adaptive, adaptive behaviour.
- Handles dynamic changes (e.g., rerouting in real time).
- Scales well for large, complex networks.
- Emergent global optimisation from local agent interactions.

Example:

- ACO dynamically adjusts pheromone trails based on updated travel times, ensuring near-optimal delivery routes even under unpredictable conditions.

Mathematical Foundations of CI

- **Fitness Landscapes:** Represent solution quality as a function:

$$f(x): \mathbb{R}^n \rightarrow \mathbb{R},$$

where $\mathbf{x}$ is a candidate solution and $f(\mathbf{x})$ its fitness.

- **Stochastic Search:** Randomised operators for exploration:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2),$$

where ϵ represents a random perturbation and σ is the standard deviation of the normal distribution $\mathcal{N}(0, \sigma^2)$. The mutation and probabilistic updates ensure diversity.

- **Complexity Insight:** Metaheuristics approximate solutions for NP-hard problems:

Time Complexity (EA) $\approx O(n \cdot g)$,

Time Complexity (Swarm) $\approx O(n \cdot d)$

Where **EA** represents the Evolutionary Algorithms, **n** the population size, **g** the generations and **d** the problem dimension.

- **EA belong to a family** of nature-inspired metaheuristic optimisation techniques that mimic biological evolution. They work by maintaining a population of candidate solutions and applying operators such as:
 - Selection: Prefer fitter individuals.
 - Crossover (Recombination): Combine parts of two solutions.
 - Mutation: Introduce random changes for diversity.
 - Replacement: Form the next generation.

Major CI Research Areas

- Evolutionary Computation
- Swarm Intelligence
- Neural Networks
- Fuzzy Logic Systems
- Reinforcement Learning

Metaheuristics Taxonomy and Performance Metrics

Metric	PSO (Particle Swarm)	GA (Genetic Algorithm)	ACO (Ant Colony)
Convergence Speed	Fast initial convergence; risk of stagnation	Slower but thorough global search	Moderate; depends on pheromone update
Scalability	Good for continuous spaces	Effective for mixed-variable problems	Handles discrete/graph-based problems well
Robustness	Sensitive to parameter tuning	High diversity; robust under noise	Adaptive to dynamic environments

Nature-Inspired Optimisation

Nature-Inspired Optimisation (NIO) algorithms are *metaheuristic techniques* that emulate processes observed in nature (such as evolution, swarm behaviour, or physical phenomena) to search for optimal or near-optimal solutions in complex problem spaces.

Traditional mathematical optimisation often fails when the problem is:

- Non-linear or non-differentiable
- Multimodal (many local minima)
- Dynamic or uncertain
- Defined by incomplete data: NIO overcomes these by *using randomness, adaptation, and distributed intelligence*.

Swarm Intelligence

Swarm Intelligence (SI) studies how simple agents interacting locally can collectively exhibit intelligent global behaviour.

Key Principles:

- Decentralised control
- Self-organisation
- Emergent behaviour
- Stochastic exploration

Applications: Optimisation, scheduling, routing, clustering, and feature selection.

Common Algorithms:

- Particle Swarm Optimisation (PSO)
- Ant Colony Optimisation (ACO)
- Artificial Bee Colony (ABC)

Particle Swarm Optimisation (PSO)

Inspired by bird flocking and fish schooling. Each *particle* represents a potential solution. Particles move in the search space, influenced by:

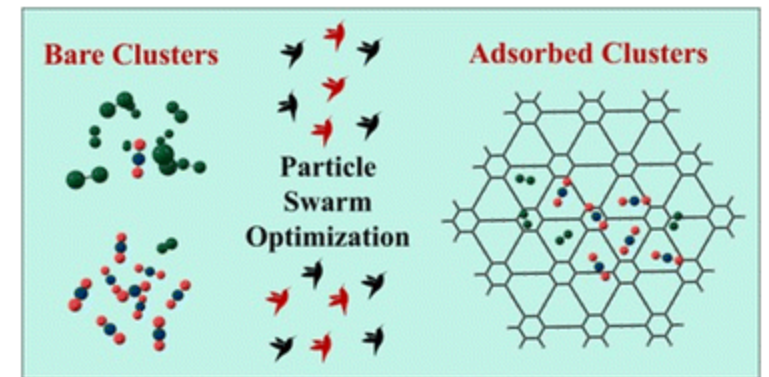
- **pbest**: their personal best solution
- **gbest**: the global best solution found so far

Key Strengths:

- Simple, few parameters, and efficient for continuous optimisation.
- Balances exploration (searching new areas) and exploitation (refining good solutions).

Applications:

- Tuning neural networks
- Engineering design
- Environmental optimisation



Ant Colony Optimisation (ACO)

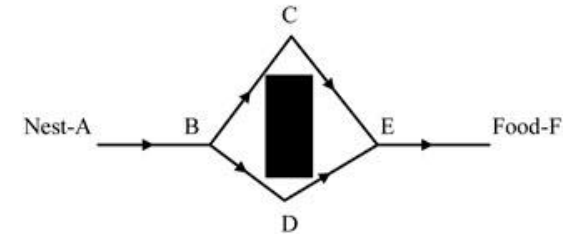
- Inspired by how ants find the shortest paths between food and the nest.
- Ants deposit **pheromones** on paths; more pheromones attract more ants → positive feedback.

Algorithm Steps:

- Initialise pheromone trails.
- Construct solutions probabilistically based on pheromone intensity.
- Evaluate each solution's quality.
- Update pheromones (reinforce good paths, evaporate poor ones).

Applications:

- Travelling Salesman Problem (TSP) - select the shortest possible route a salesman must take to visit a given set of cities exactly once and return to the starting city.
- Network routing
- Resource allocation



Artificial Bee Colony (ABC)

Mimics the foraging behaviour of honeybee swarms.

Bee types:

- **Employed bees:** exploit food sources (solutions).
- **Onlooker bees:** choose food sources based on shared information (fitness).
- **Scout bees:** explore new areas when old ones are exhausted.

Algorithm Steps:

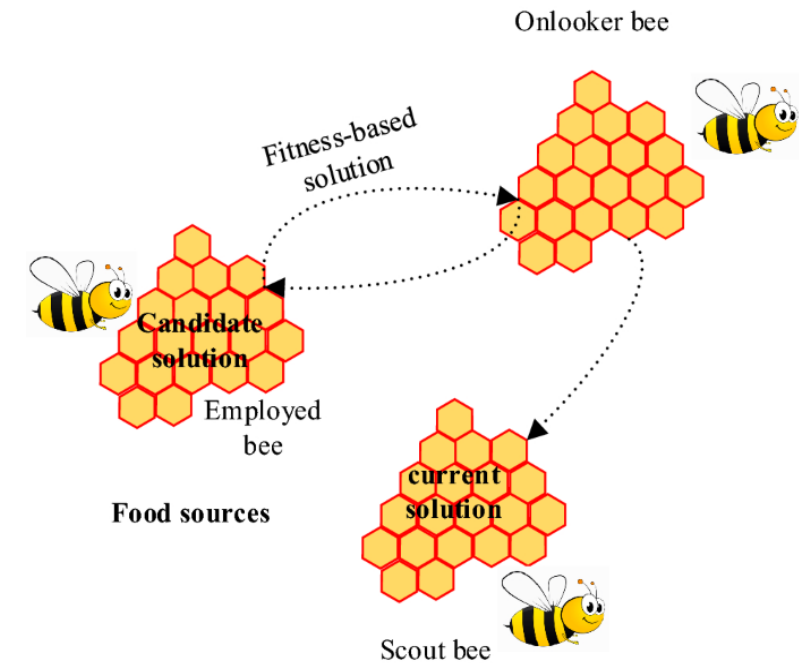
- Initialise food sources randomly.
- Employed bees explore neighbourhoods of current solutions.
- Onlookers select and refine promising sources.
- Scouts replace abandoned sources.

Strengths:

- Strong exploration capability.
- Effective for discrete, continuous, and constrained problems.
- Simple to hybridise with other CI methods.

Applications:

- Scheduling and load balancing
- Parameter optimisation
- Image segmentation and feature selection



Comparison Summary

Feature	PSO	ACO	ABC
Inspiration	Bird flocking	Ant foraging	Bee foraging
Agents	Particles	Ants	Bees
Mechanism	Social learning	Pheromone trails	Information sharing
Best for	Continuous optimisation	Discrete/graph problems	Mixed or multimodal search
Strengths	Fast convergence	Path discovery	Exploration
Weaknesses	May stagnate	Parameter-sensitive	Slower exploitation

Evolutionary Computation: Genetic Algorithms

Inspired by Darwinian evolution. Population of candidate solutions evolves through *selection*, *crossover*, and *mutation*. Each individual represents a possible solution encoded as a “chromosome”.

Algorithm Outline:

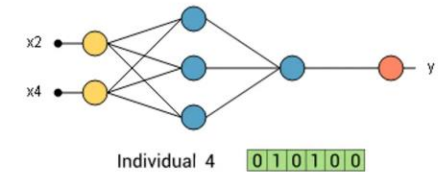
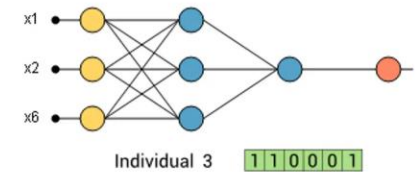
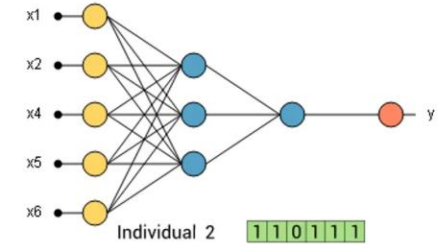
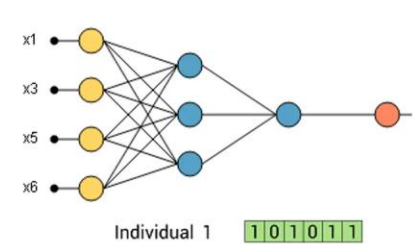
- Initialise a random population of chromosomes.
- Evaluate fitness for each individual.
- Select parents based on fitness.
- Apply crossover to create offspring.
- Mutate offspring to maintain diversity.
- Replace old population with new and repeat.

Strengths:

- Global search capability.
- Works with discrete, continuous, or mixed variables.
- Robust to local minima.

Limitations:

- Parameter sensitivity (population size, mutation rate).
- Can converge slowly.



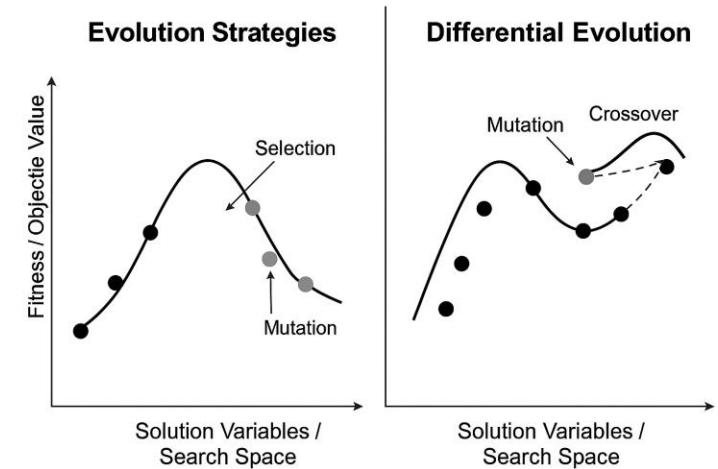
Evolution Strategies and Differential Evolution

Evolution Strategies (ES):

- Focus on **self-adaptive mutation** and **real-valued parameters**.
- Emphasises mutation over crossover.

Differential Evolution (DE):

- Uses differences between individuals for mutation.



Method	Representation	Strength	Common Use
ES	Real-valued	Adaptive step-size	Engineering optimisation
DE	Vector-based	Fast convergence	Continuous and hybrid systems

Artificial Neural Networks

Modelled after biological neurons. Networks consist of **layers** of interconnected nodes (neurons). Each connection has a **weight** representing learned importance.

- **Training Process:**

- Forward propagation – compute output.
- Compute error between prediction and target.
- Backpropagation – update weights using gradient descent.

- **Common Architectures:**

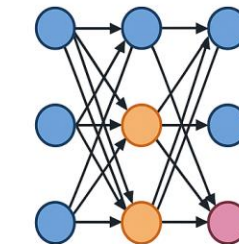
- Feedforward Neural Network (FNN)
- Convolutional Neural Network (CNN)
- Recurrent Neural Network (RNN)

- **Advantages:**

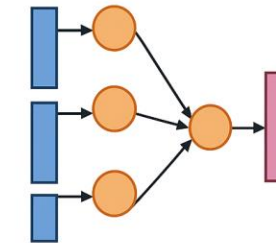
- Universal function approximators.
- Excellent for classification and regression.

- **Limitations:**

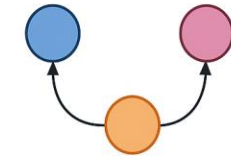
- Require large datasets and computational power.
- Limited interpretability (“black box”).



Feedforward
Neural Network
(FNN)



Convolutional
Neural Network
(CNN)



Recurrent
Neural Network
(RNN)

Spiking Neural Networks

- Spiking Neural Networks (SNNs) mimic the **timing-based information processing** of the brain.
- Unlike traditional ANNs that use continuous activations, SNNs communicate via **discrete electrical pulses (spikes)**.
- Information is encoded in **spike timing, frequency, or population dynamics**.

Key Features:

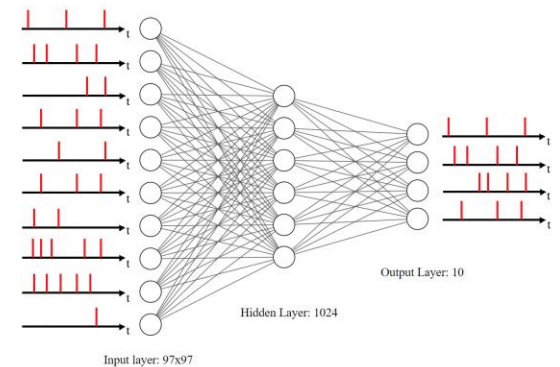
- **Temporal Coding:** The *time* a neuron spikes carries information.
- **Event-driven computation:** Neurons remain idle until stimulated (energy-efficient).
- **Biological Plausibility:** Mimics neuron membrane potential and synaptic plasticity.

Learning Rules:

- **Spike-Timing-Dependent Plasticity (STDP):** Strengthens synapses when pre-synaptic spikes precede post-synaptic spikes.

Applications:

- Neuromorphic chips (e.g., *Intel Loihi*, *SpiNNaker*).
- Event-based vision sensors (Dynamic Vision Sensors).
- Real-time acoustic and tactile processing.



Fuzzy Logic Systems

Classical logic is binary (0 or 1), but fuzzy logic allows **partial truth values** between 0 and 1. Inspired by human reasoning — useful when dealing with uncertainty or linguistic variables (“high temperature”, “slow speed”).

Key Components:

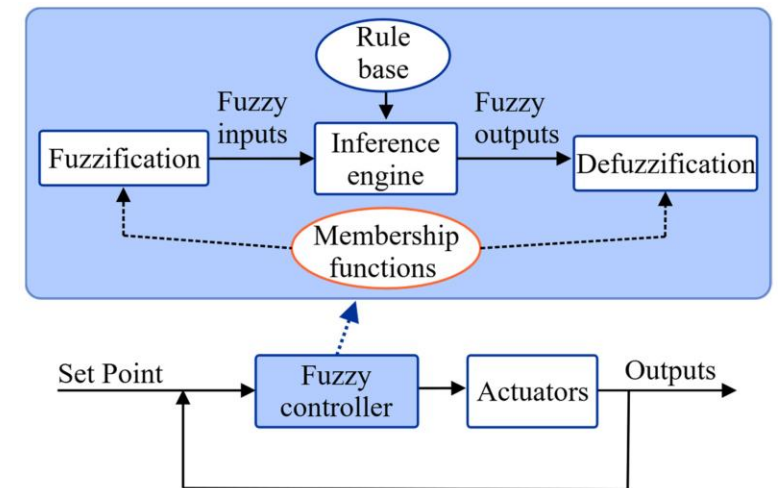
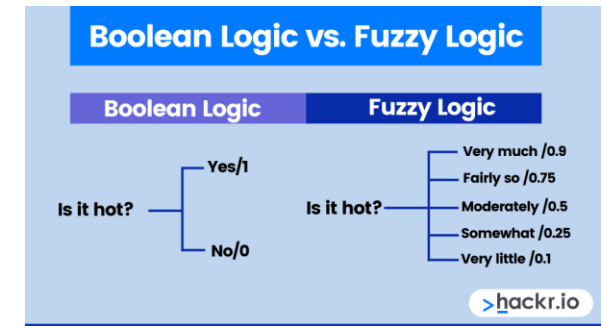
- **Fuzzification:** Convert crisp input values into fuzzy sets using membership functions.
 - Example:
 - Temperature = 30°C → “Medium Hot” with $\mu = 0.7$
- **Inference Engine:** Apply fuzzy rules (IF–THEN).
 - IF temperature is “High” AND humidity is “Low” → THEN fan speed is “Fast”
- **Defuzzification:** Convert fuzzy output back into a crisp value (e.g., centroid method).

Advantages:

- Works well with imprecise, noisy, or linguistic data.
- Easy to interpret and implement.
- Excellent for control systems and decision-making.

Applications:

- Washing machines, cameras, and climate control systems.
- Financial risk assessment.
- Hybrid AI models (Fuzzy-Neural Systems).



Algorithmic Complexity & Hybridisation Strategies

1. Time Complexity (Higher population or dimensionality → increased computational cost)

- Evolutionary Algorithms (EA): $O(n \cdot g)$, where n is the population size, g is the number of generations
- Swarm Algorithms (PSO, ACO): $O(n \cdot d)$, where d is the problem dimensionality.

2. Parameter Sensitivity

- PSO: Sensitive to inertia weight (w), cognitive (c_1) and social (c_2) coefficients. Poor tuning → stagnation or divergence.
- GA: Mutation rate and crossover probability strongly affect convergence speed and diversity. No universal parameter set; tuning often problem-specific.

3. Hybrid Approaches (PSO + GA)

- Why Hybridise? - Combine PSO's fast convergence with GA's diversity maintenance.

Example Strategy:

- Use PSO for global search → quick approximation.
- Apply GA operators (crossover, mutation) to refine solutions and avoid premature convergence.

Benefits: Improved robustness, better handling of multimodal landscapes, and adaptability in dynamic environments.

Hybrid and Neuro-Evolutionary Systems

Hybrid systems integrate two or more CI techniques (e.g., **neural networks**, **evolutionary algorithms**, **fuzzy logic**) to overcome individual limitations.

- **Common Types of Hybrid Systems:**

- **Neuro-Fuzzy Systems (ANFIS):**

- Neural networks learn fuzzy rules automatically.
 - Example: Adaptive Neuro-Fuzzy Inference System.

- **Neuro-Evolutionary Systems:**

- Evolutionary algorithms optimise network weights, topologies, or hyperparameters.
 - Example: NeuroEvolution of Augmenting Topologies (NEAT).

- **Fuzzy-Genetic Systems:**

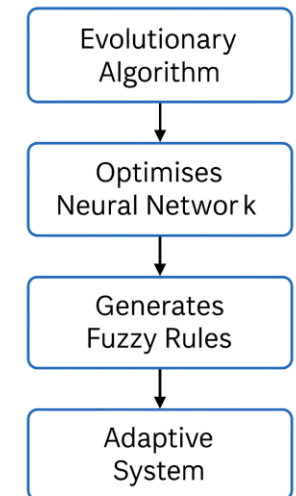
- Genetic Algorithms evolve membership functions and fuzzy rule sets.
 - **Benefits:**

Combines **interpretability (fuzzy)** with **learning ability (neural)** and **global optimisation (evolutionary)**. Handles **dynamic, complex**, or **multi-objective** problems.

Applications:

- Adaptive control systems.
- Real-time fault detection and predictive maintenance.
- Financial forecasting, environmental modelling, and robotics.

Concept	Inspiration	Strength	Typical Use
SNNs	Biological spiking neurons	Real-time temporal processing	Neuromorphic computing, sensors
Fuzzy Logic	Human reasoning	Interpretability, uncertainty handling	Control, decision systems
Hybrid Systems	Multi-paradigm CI	Adaptability and flexibility	Intelligent automation, predictive systems



Discussion: Choosing the Right CI Paradigm

Q1: How would you decide between using an evolutionary algorithm, a swarm-based model, or a reinforcement learning agent for a given optimisation problem?

Q2: Can emergent behaviour ever become unpredictable or unsafe in autonomous systems?

Q3: What are the ethical implications of delegating decision-making to collective or adaptive AI systems?



Summary: CI for Sustainable Futures

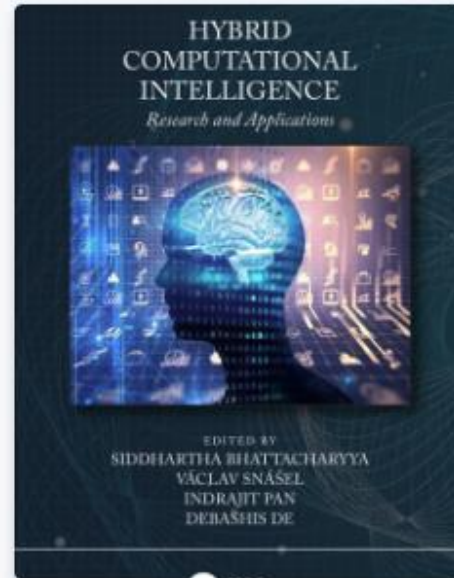
CI systems embody **learning, adaptability, and decentralised intelligence**, mirroring natural ecosystems. The next frontier: **eco-intelligent systems** that optimise not just performance, but environmental and social impact.

Key Trends:

- 1. Hybrid CI–ML Models:** Combining symbolic reasoning and deep learning.
- 2. Neuromorphic and Edge AI:** Energy-efficient computation.
- 3. Collective Intelligence at Scale:** Swarm and multi-agent systems for global monitoring.
- 4. AI for Environmental Stewardship:** Real-time ecological modelling and conservation.

“As nature inspires computation, computation can now sustain nature.”

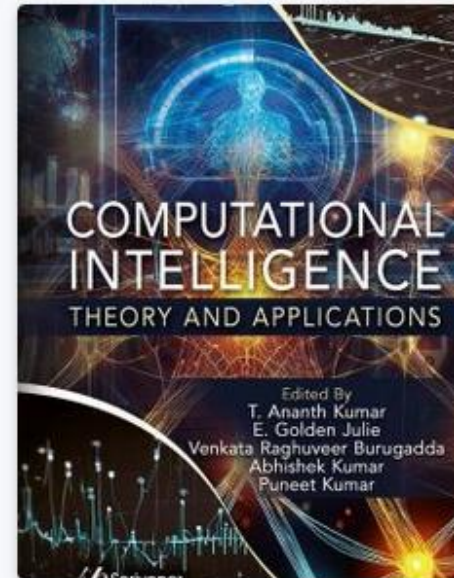
References



eBook

...

**Hybrid Computational
Intelligence**



eBook

...

Computational Intelligence



Nottingham Trent
University

Department of Computer Science

Summary

In this lecture you learnt:

- What is Computational Intelligence (CI) and how it relates to AI
- Core principles of CI
- Nature-Inspired Optimisation
- Swarm Intelligence (PSO, ACO, ABC)
- Evolutionary Computation (GA, ES, DE)
- Hybrid CI concepts
- Discussion of real-world optimisation scenarios

This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 1: Introduction to Computational Intelligence and Nature-Inspired Algorithms

Dr Pedro Machado pedro.machado@ntu.ac.uk